Nama : Ananta Putri Clara
Kelas : 2024B
NIM : 24031554031

UTS GSD

2.1 Wrangling dan Exploratory Data Analysis

1. Penentuan Periode Waktu
   Dataset dibagi menjadi dua periode yang tidak saling tumpang tindih, yaitu P1 sebagai data pelatihan (training) dan P2 sebagai data pengujian (testing)

Periode	**Rentang Waktu**	**Fungsi**
P1	1-15 Agustus 2015	Training
P2	16-31 Agustus 2015	Testing

   Pembagian secara kronologis dilakukan agar pola interaksi pada periode awal dapat digunakan sebagai dasar rekomendasi, sedangkan interaksi pada periode berikutnya digunakan untuk mengevaluasi hasil rekomendasi.

2. Filter timestamp dan penyimpanan CSV
   Atribut timestamp yang semula menggunakan format unix epoch dalam milidetik dikonversi menjadi tipe datetime menggunakan fungsi pd.to_datetime().

   Data kemudian difilter sesuai rentang P1 dan P2. Batas akhir periode diterapkan secara eksklusif agar seluruh interaksi pada tanggal terakhir tetap tercakup tanpa memasukkan data dari periode berikutnya.

   Hasil penyaringan menunjukkan:

Keterangan	**P1 (Training)**	**P2 (Testing)**
Jumlah interaksi	265.509	287.853
Timestamp pertama	2015-08-01 00:00:14.822	2015-08-16 00:00:01.663
Timestamp terakhir	2015-08-15 23:59:52.844	2015-08-31 23:59:58.647

   Hasil filter disimpan dalam dua file terpisah, yaitu P1_training.csv dan P2_testing.csv. Kedua file digunakan untuk tahap analisis berikutnya.

3. Komposisi subgraf menggunakan Python API
   Komposisi subgraf diperoleh melalui query Cypher yang dijalankan menggunakan Python Driver. Query tersebut mencocokkan pola (v:Visitor)-[r]->(i:Item), kemudian membatasi relationship berdasarkan timestamp pada masing-masing periode.

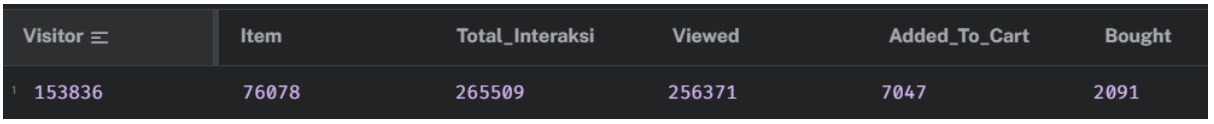
Fungsi count(DISTINCT v) digunakan untuk menghitung jumlah Visitor unik, sedangkan count(DISTINCT i) menghitung jumlah item unik. Selanjutnya, count(r) menghitung keseluruhan interaksi, dan CASE WHEN digunakan untuk memisahkan jumlah berdasarkan jenis relationship.

Hasil pengujian melalui Python API adalah sebagai berikut:

Komponen	**P1**	**P2**
Visitor unik	153.836	163.714
Item unik	76.078	79.399
Total interaksi	265.509	287.853
VIEWED	256.371	277.534
ADDED_TO_CART	7.047	7.778
BOUGHT	2.091	2.541

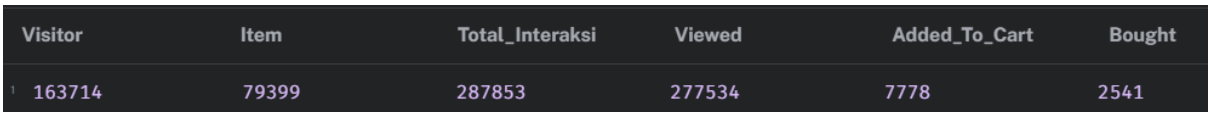
Berdasarkan hasil tersebut, relationship VIEWED mendominasi kedua periode. Hal ini menunjukkan bahwa aktivitas melihat produk terjadi jauh lebih sering dibandingkan penambahan produk ke keranjang maupun transaksi pembelian.

Jumlah Visitor, Item, dan interaksi pada P2 juga lebih tinggi dibandingkan P1. Hasil query Neo4j sesuai dengan ringkasan data yang diperoleh melalui Pandas sehingga mendukung konsistensi proses filter dan impor data



Visitor	Item	Total_Interaksi	Viewed	Added_To_Cart	Bought
153836	76078	265509	256371	7047	2091

Gambar 1. Komposisi P1



Visitor	Item	Total_Interaksi	Viewed	Added_To_Cart	Bought
163714	79399	287853	277534	7778	2541

Gambar 2. Komposisi P2

4. Vector Space Modelling (VSM) terhadap User
   VSM terhadap user dilakukan dengan membentuk matriks User-Item berdasarkan interaksi pada periode P1. Setiap baris merepresentasikan seorang user, sedangkan setiap kolom merepresentasikan sebuah item.

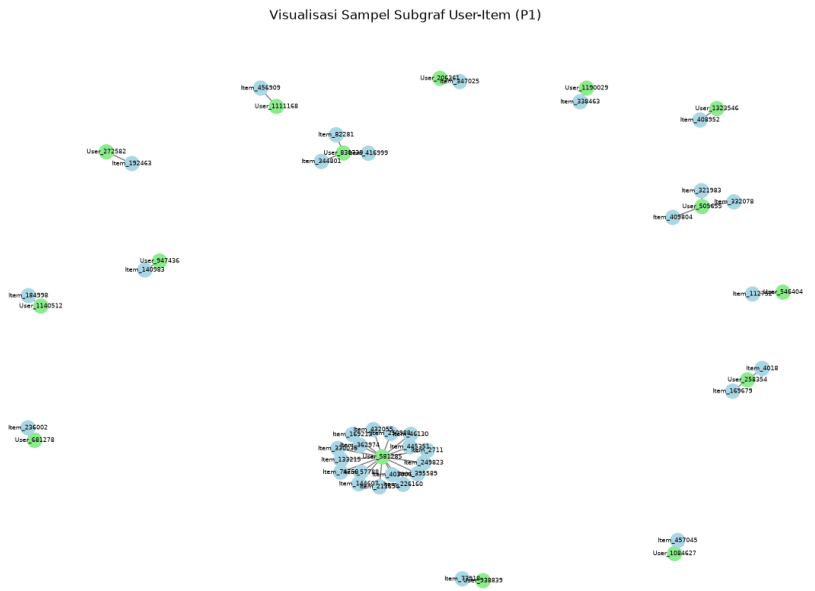
   Nilai pada matriks menunjukkan frekuensi interaksi antara user dan item. Apabila user berinteraksi beberapa kali dengan item yang sama, nilainya akan terakumulasi. Dengan demikian, matriks yang digunakan bukan sekedar matriks biner 0 dan 1.

Matriks dibentuk menggunakan coo_matrix dan dikonversi menjadi format CSR (Compressed Sparse Row) agar penyimpanan data yang didominasi nilai nol menjadi lebih efisien. Hasil pembentukan matriks:

Karakteristik	**Hasil**
Jumlah user	153.836
Jumlah item	76.078
Shape matriks	(153836, 76078)
Pasangan user-item unik (nonzero entries)	213.066
Dimensi setiap vektor user	76.078

Artinya, setiap user direpresentasikan sebagai sebuah vektor berdimensi 76.078, sesuai jumlah item unik pada P1. Nilai nonzero sebesar 213.066 menunjukkan banyaknya pasangan user-item yang memiliki interaksi, bukan jumlah seluruh kejadian interaksi.

Untuk mempermudah pengamatan, dibuat visualisasi menggunakan NetworjX dengan mengambil sampel 15 user pertama beserta item yang berinteraksi dengan mereka.



Gambar 3. Visualisasi VSM User

Visualisasi tersebut memperlihatkan hubungan antara user dan item. User yang terhubung dengan beberapa item menunjukkan bahwa seorang pengunjung dapat memiliki lebih dari satu riwayat interaksi. Sebaliknya, item yang terhubung dengan beberapa user memperlihattkan adanya produk yang diakses oleh lebih dari satu pengunjung.

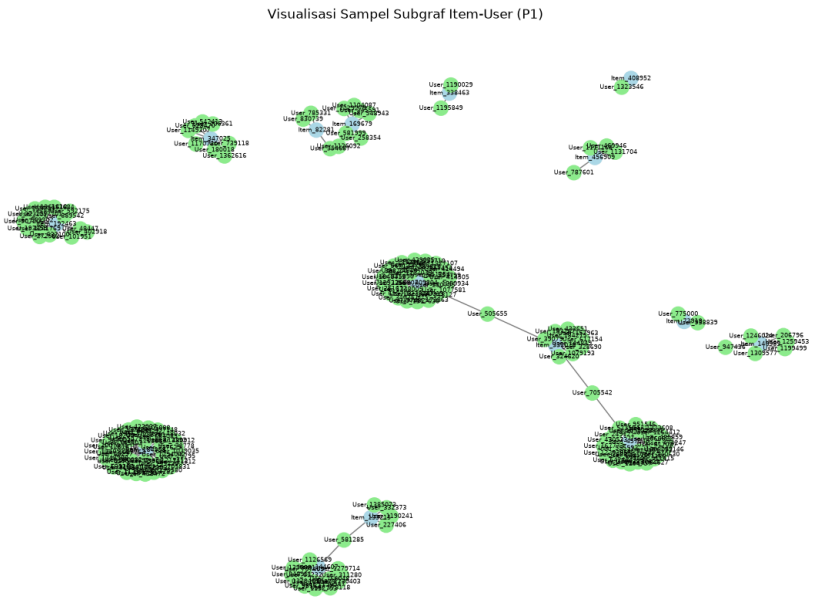
5. Vector Space Modelling (VSM) terhadap Item
   VSM terhadap item dibentuk dengan melakukan transpose pada matriks user-item sehingga diperoleh matriks item-user. Pada representasi ini, setiap baris menunjukkan satu item dan setiap kolom menunjukkan satu user.

   Nilai matriks tetap menunjukkan frekuensi interaksi, tetapi analisis yang dilihat berubah dari perilaku user terhadap item menjadi karakteristik item berdasarkan user yang mengaksesnya. Hasil pembentukan matriksnya:

Karakteristik	**Hasil**
Jumlah item	76.078
Jumlah user	153.836
Shape matriks	(76078, 153836)
Pasangan item-user unik	213.066
Dimensi setiap vektor item	153.836

   Setiap item direpresentasikan oleh vektor berdimensi 153.836. Jumlah pasangan unik tidak berubah karena matriks item-user merupakan hasil transpose dari matriks user-item.

   Visualisasi NetworkX dilakukan menggunakan sampel 15 item pertama beserta user yang memiliki interaksi dengan item tersebut



   Gambar 4. Visualisasi VSM Item
   Melalui representasi ini, 2 item dapat dibandingkan berdasarkan pola interaksi pengunjungnya. Item yang diakses oleh user yang sama memiliki potensi

kemiripan perilaku, sehingga pendekatan ini dapat digunakan sebagai dasar perhitungan Item-Based Similarity

6. Perbandingan Efisiensi Proyeksi User-Based dan Item-Based

   Perbandingan dilakukan berdasarkan jumlah kandidat pasangan yang perlu diperiksa apabila seluruh user atau seluruh item dibandingkan satu sama lain. Jumlah kandidat pasangan dihitung menggunakan rumus:

$$N = \frac{n(n-1)}{2}$$

   dengan n merupakan jumlah objek yang dibandingkan. Hasil perhitungannya:

Keterangan	**User-Based**	**Item-Based**
Jumlah Objek	153.836	76.078
Jumlah kandidat pasangan	11.836.680.530	2.893.893.003

   Berdasarkan hasil perhitungan, rasio jumlah kandidat pasangan User-Based terhadap Item-Based adalah sekitar 4,09 kali. Oleh karena itu, Item-Based dipilih untuk tahap perhitungan similarity pada No. 7 karena memiliki jumlah kandidat pasangan yang lebih sedikit. Hal ini menunjukkan potensi pengurangan beban komputasi apabila seluruh pasangan akan dibandingkan.

7. Perhitungan Similarity pada Item-Based
   Berdasarkan hasil perbandingan sebelumnya, perhitungan kemiripan dilakukan menggunakan pendekatan Item-Based dengan metode Cosine Similarity.

   Cosine Similarity mengukur kemiripan dua vektor berdasarkan sudut di antara keduanya. Dalam penelitian ini, vektor item dibentuk berdasarkan frekuensi interaksi masing-masing user pada periode P1.

   Hasil akhir dalam perhitungan ini akan menghasilkan nilai dengan rentang 0 sampai 1. Jika nilai skor semakin mendekati angka 1, artinya kedua item tersebut memiliki tingkat kemiripan atau kecenderungan interaksi yang sangat tinggi. Sebaliknya, jika menunjukkan angka 0, artinya kedua vektor tidak memiliki komponen interaksi yang sama atau tidak pernah diakses oleh pengguna yang sama.

   Item acuan ditentukan berdasarkan jumlah user unik terbanyak pada P1. Hasilnya diperoleh **item 187946** dengan 1.317 user unik. Pemilihan item yang paling banyak diakses dilakukan agar item acuan memiliki riwayat interaksi yang cukup untuk dibandingkan dengan item lainnya.

a. Menampilkan lima item dengan Similarity tertinggi

Perhitungan Cosine Similarity dilakukan melalui Python menggunakan fungsi cosine.similarity() dari SCikit-learn. Item acuan dibandingkan dengan seluruh item pada matriks item-user, kemudian similarity terhadap dirinya sendiri dikeluarkan dari hasil.

Nilai similarity yang positif selanjutnya dikirim sebagai parameter ke query Cypher menggunakan UNWIND. Query mengurutkan hasil berdasarkan similarity tertinggi dan membatasi keluaran sebanyak lima record.

```
query_7a = """
UNWIND $data AS row
RETURN $acuan AS item_acuan,
       row.itemid AS item_tujuan,
       row.score AS similarity
ORDER BY similarity DESC
LIMIT 5
"""
```

Dengan hasil:

Peringkat	Item acuan	Item tujuan	Cosine similarity
1	187946	127784	0,074227
2	187946	22913	0,023689
3	187946	160665	0,023327
4	187946	38189	0,022334
5	187946	382128	0,022187

Item 127784 memiliki similarity tertinggi terhadap item acuan, yaitu sekitar 0,074227, sementara itu, empat item kainnya memiliki nilai yg lebih rendah.

Seluruh nilai tersebut relatif mendekati nol. Artinya, meskipun kelima item merupakan kandidat dengan similarity tertinggi terhadap item acuan, kemiripan pola interaksinya secara keseluruhan masih tergolong sangat rendah.

Hal ini dapat terjadi karena matriks interaksi bersifat sparse dan tidak semua item memiliki banyak user yang sama. Dengan demikian, peringkat similarity menunjukkan kedekatan relatif terhadap kandidat lain, bukan berarti kedua item memiliki kemiripan yang sangat kuat.

b. Menambahkan nilai Similarity ke Graf Neo4j

Setelah diperoleh lima item dengan similarity tertinggi, hasil perhitungan ditambahkan ke database Neo4j melalui relationship SIMILAR_TO.

```
// bukti realtionship SIMILAR_TO no 2.1(7b)
MATCH (a:Item {itemid: 187946})-[r:SIMILAR_TO]->(b:Item)
RETURN
    a.itemid AS Item_Acuan,
    b.itemid AS Item_Tujuan,
    r.score AS Cosine_Similarity
ORDER BY Cosine_Similarity DESC;
```

Query Cypher menggunakan MATCH untuk mencari item acuan dan item tujuan berdasarkan ID masing-masing. Selanjutnya, MERGE digunakan untuk membentuk relationship SIMILAR_TO, sedangkan SET r.score=$score digunakan untuk menyimpan nilai Cosine Similarity sebagai properti relationship.

Dengan demikian, hubungan similarity tidak hanya tersedia dalam bentuk tabel Python, tetapi juga tersimpan secara eksplisit dalam graf Neo4j.

	Relationship	Jumlah
1	"VIEWED"	2664312
2	"ADDED_TO_CART"	69332
3	"BOUGHT"	22457
4	"SIMILAR_TO"	5

Gambar 5. Hasil Query relationship SIMILAR_TO

Hasil menunjukkan bahwa lima relationship SIMILAR_TO berhasil dibuat antara item 187946 dan kelima item tujuannya.

8. Visualisasi Graf sebelum dan sesudah penambahan similarity
   a. Kondisi sebelum penambahan similarity

   Graf awal menampilkan sampel 15 record interaksi dari Visitor menuju item acuan 187946 pada periode P1. Relationship yang ditampilkan diibatasi pada interaksi asli, yaitu VIEWED, ADDED_TO_CART dan BOUGHT.

   Pada tahap ini, belum ditampilkan relationship similarity antara item acuan dan item lainnya.



Gambar 6. Graf sebelum penambahan similarity

Visualisasi tersebut memperlihatkan bahwa struktur graf awal berfokus pada hubungan antara pengunjung dan produk. Informasi yang dapat diamati masih berupa aktivitas pengunjung terhadap item acuan, belum mencakup hubungan kemiripan antarproduk.

b. Kondisi sesudah penambahan similarity
Pada tahap berikutnya, lima relationship SIMILAR_TO ditambahkan ke graf awal. Relationship ini menghubungkan item 187946 dengan lima item yang memiliki nilai Cosine Similarity tertinggi.



Gambar 7. Graf sesudah penambahan similarity

Setelah penambahan similarity, graf tidak hanya menggambarkan hubungan Visitor-Item, tetapi juga membuat hubungan item-item berdasarkan pola interaksi user.

Penambahan relationship tersebut memperkaya informasi pada graf karena item acuan sekarang memiliki hubungan langsung dengan produk lain yang teridentifikasi memiliki pola interaksi relatif paling mirip.

2.2 Analytics

1. Pemilihan user secara acak dan implementasi KNN

   Pada tahap Analytics, sistem rekomendasi dibangun menggunakan pendekatan User-Based K-Nearest Neighbors (KNN). Berbeda dengan analisis Item-Based sebelumnya, pendekatan ini mencari user lain yang memiliki pola interaksi paling mirip dengan target. Pemilihan user dilakukan secara acak dengan proses yang dapat direproduksi menggunakan np.random,default_rng(169).

   Sebelum pengacakan, diterapkan beberapa kriteria untuk memastikan user terpilih memiliki data yang memadai, yaitu

   1. Memiliki minimal lima item unik pada P1
   2. Memiliki minimal tiga item unik pada P2
   3. Memiliki minimal lima tetangga dengan Cosine Similarity positif
   4. Memiliki sedikitnya tiga kandidat item baru dari lima tetangga tersebut yang belum diakses user target pada P1.

   Kriteria pertama dan kedua menghasilkan 317 kandidat awal. Setelah pemeriksaan kelayakan berdasarkan data P1, diperoleh 94 kandidat yang valid. Pemilihan acak kemudian dilakukan terhadap daftar kandidat valid yang telah diurutkan menggunakan seed 169. Hasil pemilihannya:

Keterangan	**Hasil**
Kandidat awal	317
Kandidat valid	94
Random seed	169
User terpilih	1246166
Item unik pada P1	7
Item unik pada P2	6

   Proses ini memungkinkan pemilihan user dilakukan kembali dengan hasil yang sama selama dataset, kriteria, dan random seed tidak berubah.

Kriteria tambahan terkait tetangga dan kandidat item diperiksa menggunakan P1, bukan berdasarkan keberhasilan rekomendasi terhadap P2. Dengan demikian, data testing tidak digunakan untuk memilih user berdasarkan nilai evaluasinya.

a. Implementasi User-Based KNN
   Fungsi rekomendasi_knn() dibuat untuk menghasilkan rekomendasi berdasarkan kemiripan antar user, skrip Python untuk rekomendasi KNN:

```
# No. 2.2 (1a) — Fungsi rekomendasi User-Based KNN

def rekomendasi_knn(user_target, k, n_rekomendasi):
    # Menghitung similarity User
    idx_user = np.where(user_ids == user_target)[0][0]

    sim_user = cosine_similarity(
        user_item[idx_user],
        user_item
    ).flatten()

    sim_user[idx_user] = -1

    # Memilih k tetangga dengan similarity positif
    idx_tetangga = np.where(sim_user > 0)[0]
    idx_tetangga = idx_tetangga[
        np.argsort(-sim_user[idx_tetangga], kind="stable")
    ][:k]

    tetangga = pd.DataFrame({
        "visitorid": user_ids[idx_tetangga],
        "similarity": sim_user[idx_tetangga]
    })

    # Mengambil interaksi tetangga dari P1
    interaksi = P1[
        P1["visitorid"].isin(tetangga["visitorid"])
    ][["visitorid", "itemid"]].drop_duplicates()

    kandidat = interaksi.merge(tetangga, on="visitorid")

    # Mengecualikan Item yang sudah diakses User target
    item_sudah = set(
        P1.loc[P1["visitorid"] == user_target, "itemid"]
    )

    kandidat = kandidat[
        ~kandidat["itemid"].isin(item_sudah)
    ]

    # Menghitung skor dan mengurutkan rekomendasi
    rekomendasi = (
        kandidat.groupby("itemid", as_index=False)
        .agg(
            skor=("similarity", "sum"),
            jumlah_tetangga=("visitorid", "nunique")
        )
```

```
        .sort_values(
            ["skor", "jumlah_tetangga", "itemid"],
            ascending=[False, False, True]
        )
        .head(n_rekomendasi)
        .reset_index(drop=True)
    )

    return tetangga, rekomendasi
```

Tahapan yang dilakukan meliputi perhitungan Cosine Similarity antara user target dan seluruh user pada matriks user-item, pengecualian user target dari daftar tetangga, serta pemilihan lima user dengan nilai similarity positif tertinggi.

Setelah tetangga terdekat diperoleh, sistem mengumpulkan item yang pernah diakses oleh kelima tetangga pada P1. Item yang sudah diakses user target dikeluarkan agar rekomendasi hanya memuat kandidat item baru.

Skor rekomendasi dihitung dengan menjumlahkan nilai similarity tetangga yang berinteraksi dengan masing-masing item kandidat. Dalam implementasi notebook, setiap pasangan user-item tetangga dihitung satu kali setelah proses drop_duplicates(). Oleh karena itu, skor rekomendasi berasal dari penjumlahan similarity user yang mendukung item tersebut, bukan penjumlahan frekuensi seluruh kejadian interaksi.

Item kemudian diurutkan berdasarkan skor tertinggi, jumlah tetangga pendukung, dan ID item sebagai penentu urutan apabila terjadi kesamaan skor.

b. Tiga item rekomendasi dengan k=5
Dengan parameter k=5, diperoleh lima user dengan cosine similarity tertinggi terhadap user 1246166. Nilai similarity dihitung berdasarkan kemiripan vektor interaksi user pada periode P1.

Peringkat	**Visitor ID**	**Cosine Similarity**
1	1321193	0,636146
2	446675	0,588348
3	354499	0,585369
4	925847	0,554700
5	1069776	0,554700

Riwayat interaksi kelima tetangga digunakan untuk menentukan kandidat rekomendasi. Item yang sebelumnya telah diakses oleh user target dikeluarkan

dari daftar kandidat. Skor setiap item dihitung melalui penjumlahan nilai similarity dari tetangga yang pernah berinteraksi dengan item tersebut.

Berdasarkan interaksi kelima tetangga pada P1, diperoleh tiga rekomendasi berikut.

Peringkat	Item ID	Skor Rekomendasi	Jumlah tetangga pendukung
1	256706	1,221516	2
2	128745	0,585369	1
3	162139	0,585369	1

Hasilnya, item 256706 menempati peringkat pertama dengan skor 1,221516 dan didukung oleh dua tetangga. Sementara itu, item 128745 dan 162319 masing-masing memperoleh skor 0,585369 dengan satu tetangga pendukung.

Dengan demikian, item 256706 menjadi rekomendasi utama karena memiliki akumulasi bobot similarity tertinggi. Skor rekomendasi berbeda dari nilai Cosine Similarity tunggal karena merupakan hasil penjumlahan bobot dari beberapa tetangga.

2. Evaluasi hasil rekomendasi

Evaluasi dilakukan untuk mengetahui apakah hasil rekomendasi menggunakan metode User-Based KNN sesuai dengan interaksi aktual user pada P2. User yang dievaluasi adalah 1246166, yaitu user yang terpilih pada tahap sebelumnya.

Item rekomendasi diperoleh dari hasil perhitungan KNN menggunakan data P1. Sementara itu, item aktual diperoleh dengan mengambil seluruh item unik yang tercatat dalam riwayat interaksi unser 1246166 pada dataset P2, yaitu periode 16-31 Agustus 2015.

Berdasarkan hasil pengolahan data menggunakan Python, diperoleh:

Item rekomendasi (P1)	Item aktual (P2)
2567007	66933
128745	183176
162139	256706
-	271887
-	405862
-	457814

Dari hasil perbandingan tersebut, terdapat satu item rekomendasi yang juga muncul dalam riwayat interaksi aktual P2, yaitu item 256706. Item tersebut menempati peringkat pertama dalam daftar rekomendasi.

Dengan demikian, urutan relevansi ketiga rekomendasi adalah [1,0,0], dengan nilai 1 menunjukkan item yang sesuai dengan data aktual P2 dan nilai 0 menunjukkan item yang tidak sesuai. Hasil perbandingan ini selanjutnya digunakan sebagai dasar perhitungan dua metrik evaluasi, yaitu Hit@5 dan NDCG@5.

a. Perhitungan Hit@5

   Hit@5 digunakan untuk mengetahui apakah ada minimal satu item yang relevan dalam lima posisi teratas daftar rekomendasi. Nilai Hit@5 1 apabila ditemukan setidaknya satu item yang sesuai dengan data aktual dan 0 apabila tidak ditemukan kecocokan.

   Berdasarkan hasil evaluasi, item 256706 berada pada peringkat pertama rekomendasi jug ditemukan pada data aktual P2. Oleh karena itu diperoleh:

   Hit@5 = 1

   Hasil tersebut menunjukkan bahwa setidaknya sistem berhasil memberikan satu rekomendasi yang sesuai dengan riwayat interaksi aktual user pada periode pengujian.

b. Perhitungan NDCG@5

   NDGC@5 digunakan untuk mengevaluasi kualitas urutan rekomendasi dengan mempertimbangkan posisi item yang relevan. Semakin tinggi posisi item relevan dalam daftar rekomendasi, semakin besar kontribusinya terhadap nilai evaluasi.

   Berdasarkan hasil perbandingan, diperoleh urutan relevansi [1,0,0]. Artinya, hanya item pada peringkat pertama yang relevan, sedangkan dua item lainnya tidak sesuai dengan data aktual P2.

   Hasil perhitungan menggunakan Python adalah sebagai berikut:

Komponen	**Nilai**
DCG@5	1,000000
IDCG@5	2,948459
NDCG@5	0,339160

   Nilai DCG@5 menunjukkan skor berdasarkan posisi item relevan dalam daftar rekomendasi, sedangkan IDCG@5 merupakan skor ideal apabila lima posisi teratas diisi oleh item relevan. NDCG@5 diperoleh dengan membagi nilai DCG2% terhadap IDCG@5.

Berdasarkan hasil tersebut, diperoleh nilai NDCG@5 sebesar 0,339160 atau sekitar 33,92%. Hasil ini menunjukkan bahwa sistem berhasil menempatkan satu item relevan pada peringkat pertama, tetapi kualitas keseluruhan rekomendasi masih terbatas karena dua item lainnya tidak sesuai dengan interaksi aktual.

Meskipun metrik yang digunakan adalah Hit@5 dan NDCG@5, sistem hanya menghasilkan tiga rekomendasi sesuai ketentuan. Oleh karena itu, posisi keempat dan kelima yang tidak terisi dianggap memiliki relevansi nol dalam proses evaluasi.

3. Kesimpulan

Dataset RetailRocket berhasil diimpor ke Neo4j menggunakan Python Driver dan dibagi menjadi P1 (training) serta P2 (testing). Data P1 kemudian direpresentasikan dalam bentuk matriks User-Item dan Item-User. Berdasarkan perbandingan jumlah kandidat pasangan, pendekatan Item-Based dipilih untuk menghitung Cosine Similarity. Hasilnya diperoleh lima item dengan kemiripan tertinggi terhadap item 187946 yang selanjutnya ditambahkan ke graf melalui relationship SIMILAR_TO.

Pada tahap rekomendasi, metode User-Based KNN digunakan untuk mencari lima tetangga terdekat dari user 1246166 berdasarkan interaksi pada P1. Dari proses tersebut diperoleh tiga rekomendasi, yaitu item 256706, 128745, 162139. Setelah dibandingkan dengan interaksi aktual pada P2, hanya item 256706 saja yang sesuai, sehingga diperoleh Hit@5 sebesar 1 dan NDCG@5 sebesar 0,339160. Hasil ini menunjukkan bahwa sistem berhasil menghasilkan rekomendasi yang relevan, meskipun kualitas keseluruhan rekomendasi masih dapat ditingkatkan.